

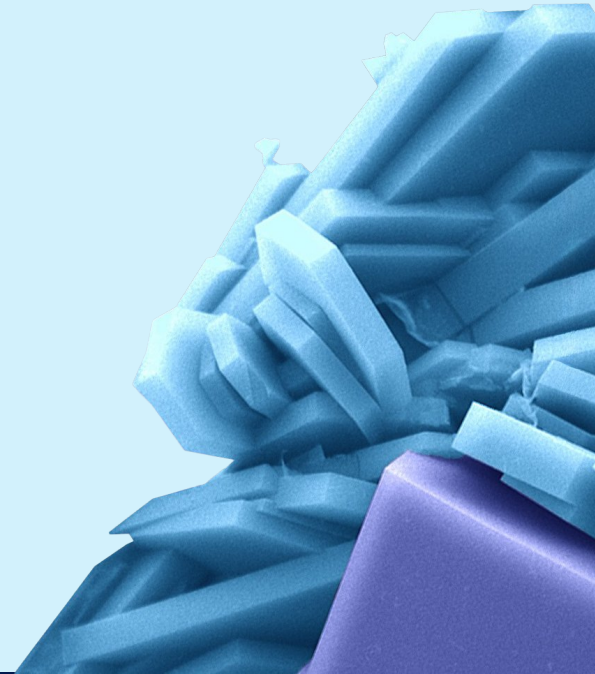
Introduction to Linux and Containerization

By Léo Orveillon



Class summary

- Introduction
- Point on Linux
- Linux for HPC
- Software environments and their issues
- Virtual Machines
- Containers
- Apptainer definitions
- Git



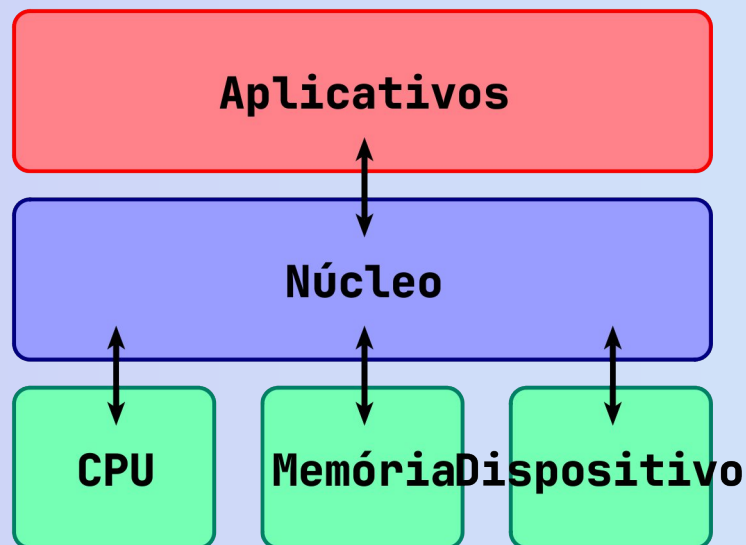
This class is

About	Not About
• Having an overview of useful tools that you'll probably have to use at some point. • Learn some basic and general concepts	• Learning complex and fundamentals Computer Science principles. • Mastering said tools

What is Linux ?

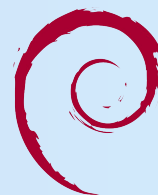
Linux is a family of **operating system** based on the **Linux Kernel**.

The kernel is the software base used by every other applications to interact with the machine



Some example of Linux operating systems

Ubuntu : Most used distribution, developed by a company, based on Debian



Debian : 2nd oldest distribution, base of many others, developed by its community

Fedora : Community distribution of a company-native distribution

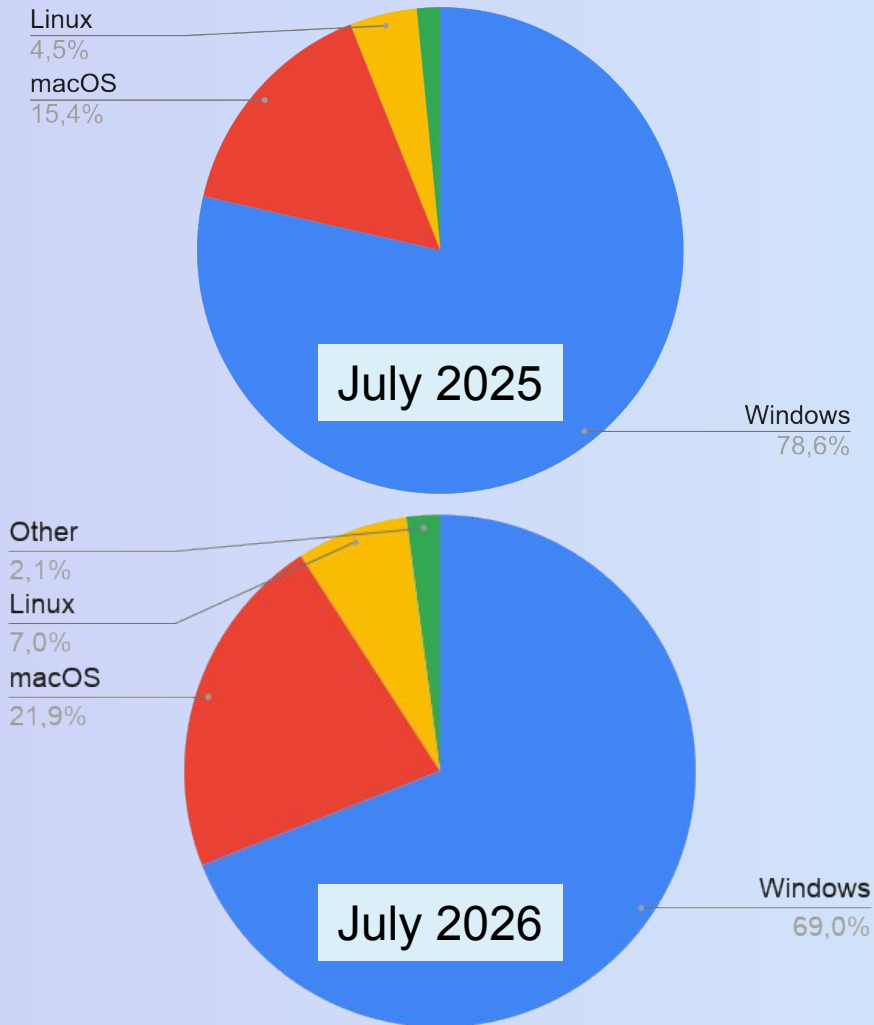


Guix : Aimed toward reproducible computational environment

Fully Open-source !
Mostly community-driven

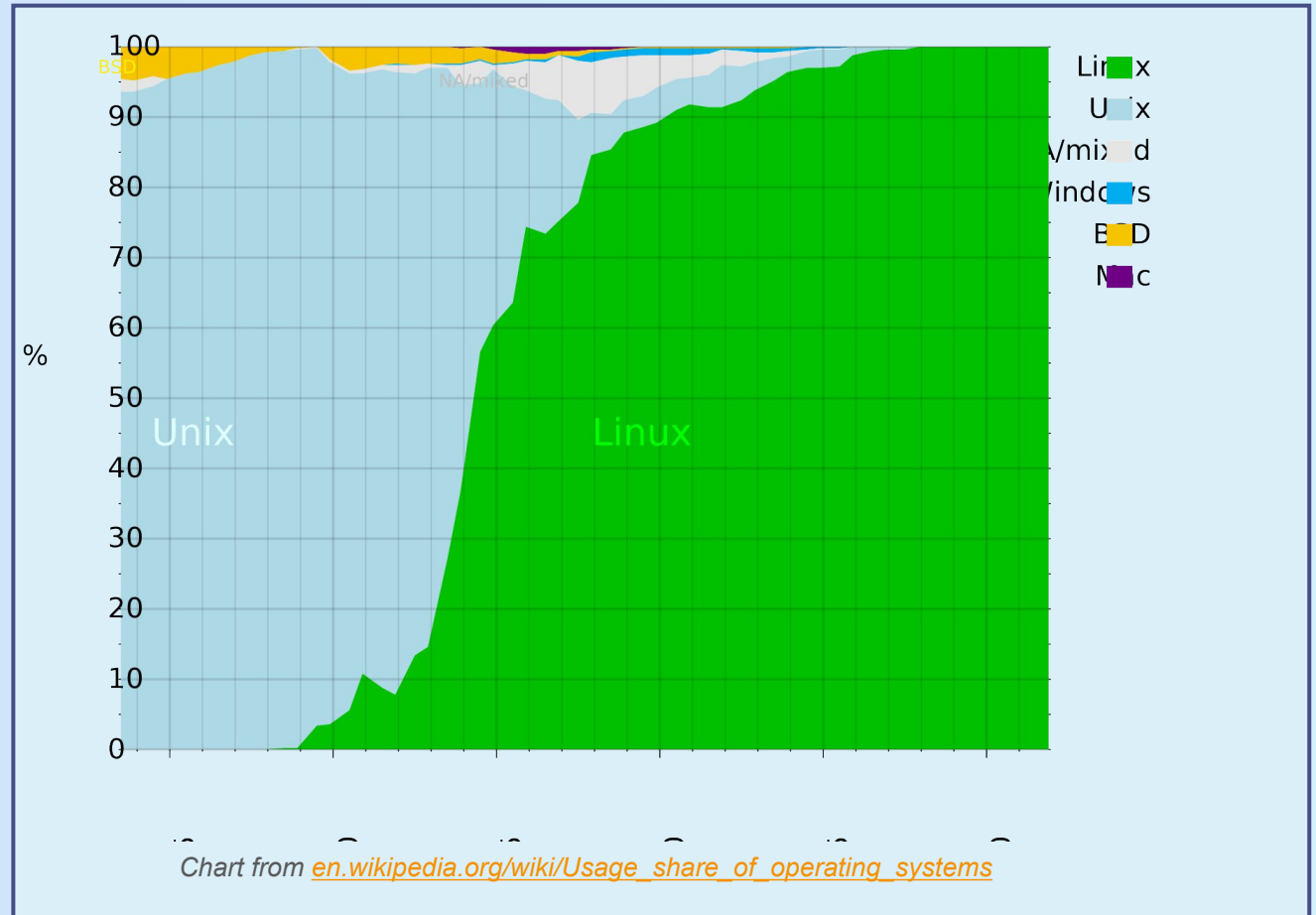
What is Linux ?

Worldwide OS distribution (Desktop computers)



Data from gs.statcounter.com/os-market-share

TOP500 OS type distribution



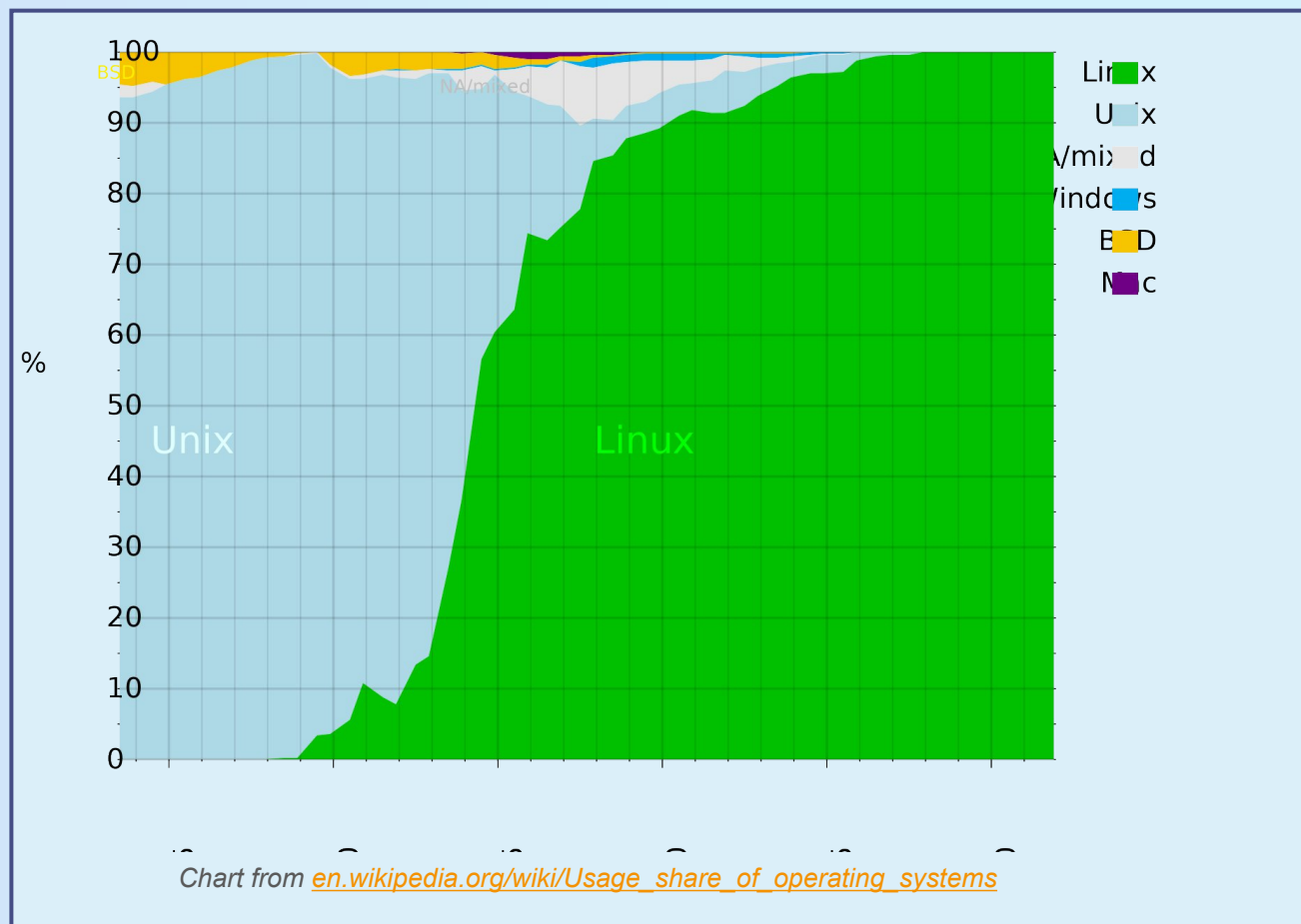
Whether you are using

- DFT
- Molecular dynamics
- CFD
- AI analysis

In your research,

You need basic linux knowledge
To make it big !

TOP500 OS type distribution



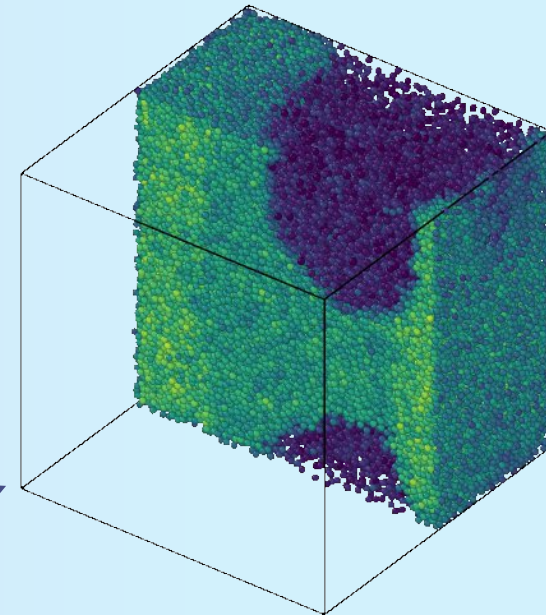
Software environments

```
File: full-run/lammps-NiAl-runscript.sh
1  #!/bin/bash
2  #OAR -l /nodes=3/cpu=2,walltime=34:30:00
3  #OAR --project pr-diamond
4
5  source /applis/site/guix-start.sh
6
7  mpirun -N 192 --hostfile $OAR_NODEFILE \
8  --bind-to core \
9  --prefix $GUIX_PROFILE \
10 --mca plm_rsh_agent "oarsh" \
11 --mca pml ucx --mca btl ^tcp,openib,uct \
12 -x UCX_TLS=shm,self,dc,rc,ud,tcp \
13 lmp_mpi -in /bettik/PROJECTS/pr-diamond/orveill/cas-noel/base_exp/in.NiAl_no-kokkos
```

instruct
to



produce



160k atoms of Al-Ni (80/20) at
1500K using LAMMPS

Steps to re-do a simulation as
described in some papers ...

Software environments

```
File: full-run/lammps-NiAl-runscript.sh
1  #!/bin/bash
2  #OAR -l /nodes=3/cpu=2,walltime=34:30:00
3  #OAR --project pr-diamond
4
5  source /applis/site/guix-start.sh
6
7  mpirun -N 192 --hostfile $OAR_NODEFILE \
8  --bind-to core \
9  --prefix $GUIX_PROFILE \
10 --mca plm_rsh_agent "oarsh" \
11 --mca pml ucx --mca btl ^tcp,openib,uct \
12 -x UCX_TLS=shm,self,dc,rc,ud,tcp \
13 lmp_mpi -in /bettik/PROJECTS/pr-diamond/orveill/cas-noel/base_exp/in.NiAl_no-kokkos
```



top layer
instructions

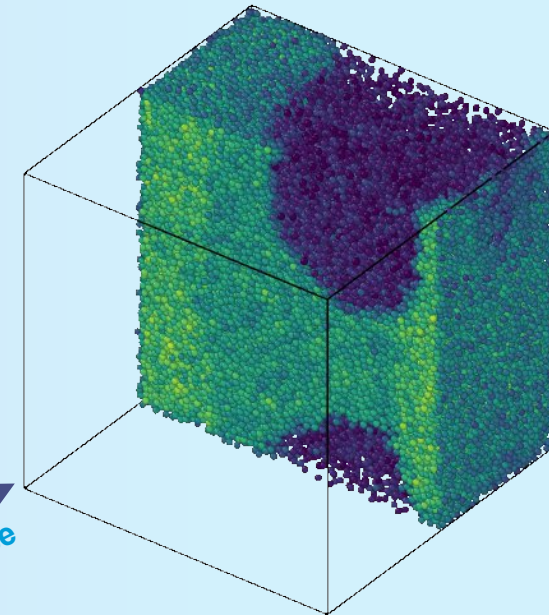
instruct to

give
feedback

pile of software



produce



160k atoms of Al-Ni (80/20) at
1500K using LAMMPS

Steps to re-do a simulation as
described in some papers ...
Are missing a **key point** !

Little to no science software work just with an OS !

Software environment

A collection of tools, libraries and configuration to develop, test and run software.



- openmpi@4.1.8
- python@3.9



- pytorch
- python-ase

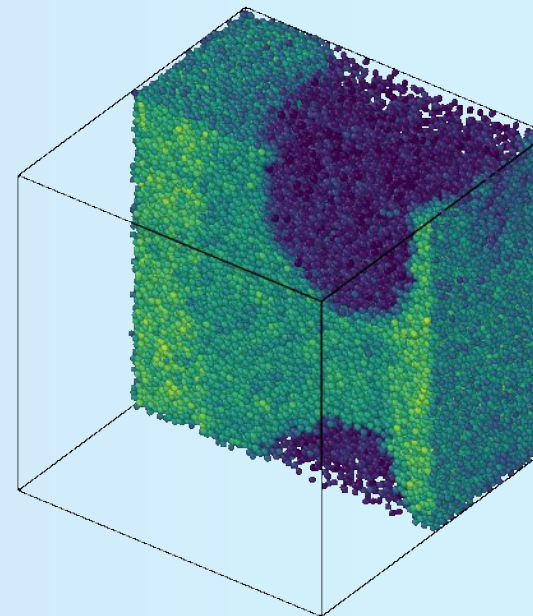
**Software environment to
run a MLIP simulation**



- lammmps@2023
- mace

Software environments

Exactly this environment
produce this exact result



160k atoms of Al-Ni (80/20) at
1500K using LAMMPS



- openmpi@4.1.8
- python@3.9



python™

- pytorch
- python-ase

**Software environment to
run a MLIP simulation**

CONDA

- lammps@2023
- mace

Software environments

Any change can impact the output

```
(pytorch stable) orveilll@bigfoot1:~$ python3 simu.py
Traceback (most recent call last):
  File "/home/orveilll/simu.py", line 1, in <module>
    import torch
  File "/applis/common/miniconda3/envs/pytorch_stable/lib/python3.10/site-packages/torch/__init__.py", line 202, in <module>
    from torch._C import * # noqa: F403
ImportError: libtorch_cpu.so: cannot enable executable stack as shared object requires: Invalid argument
(pytorch stable) orveilll@bigfoot1:~$
```



- openmpi@4.1.8
- python@3.10



python™

- pytorch
- python-ase

Software environment to run a MLIP simulation

CONDA

- lammmps@2023
- mace

In summary, we need

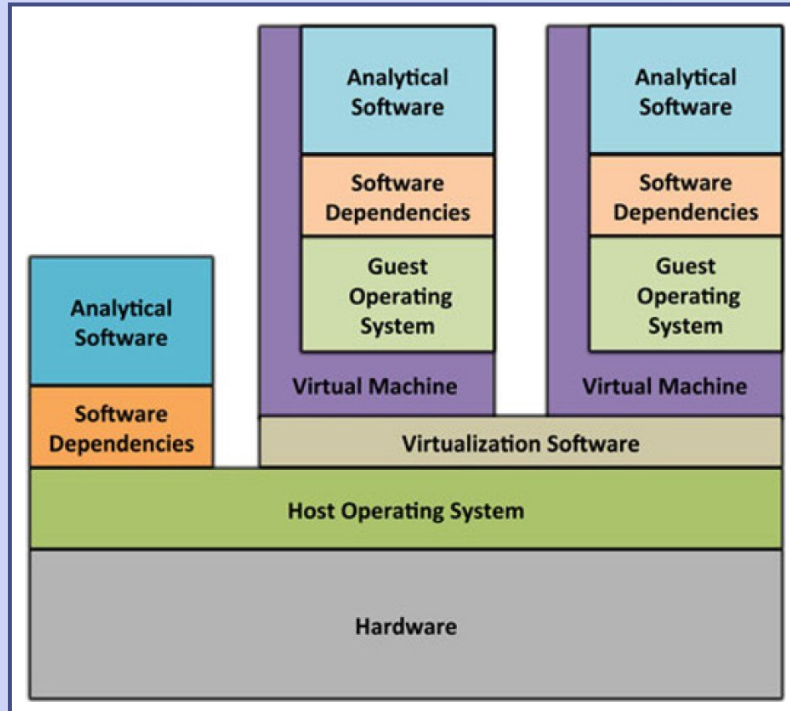
- Control and view of our software stack
- A way to share it

Because we are using shared ressources

- No interferences between users software

An **artifact** which is isolated, descriptive and shareable

Virtual Machines



Piccolo, Stephen & Frampton, Michael. (2016). Tools and techniques for computational reproducibility. GigaScience. 5. 10.1186/s13742-016-0135-4.

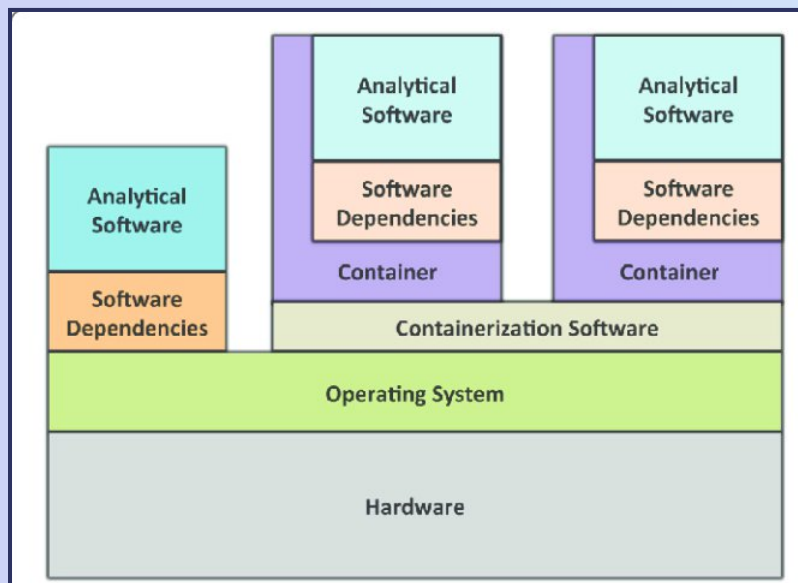
PROS

- Embedd its own Operating system
- Fully isolated from the host system
- Run the same on every host

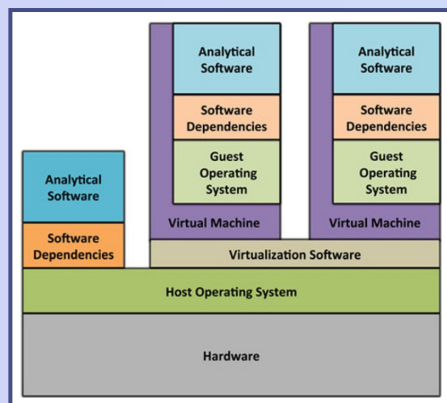
CONS

- Induce a non negligible overhead
- Not immutable (once given, user can still modify it)
- Require Linux system management knowledge

Containers



Piccolo, Stephen & Frampton, Michael. (2016). Tools and techniques for computational reproducibility. *GigaScience*. 5. 10.1186/s13742-016-0135-4.



PROS

- Negligeable overhead
- Easy to build
- Only rely on host kernel and drivers
- Immutable

CONS

- Need compatible kernel to run
- Require knowledge of containerization software commands
- Still a blackbox without the recipe

Software isolation summary

Type of isolation	Virtual Machines	Containers	Virtual environments
What's isolated	OS & Hardware	Applications & libraries	Libraries
Typical use case	Simulate a computer	Run a piece of software	Develop a project

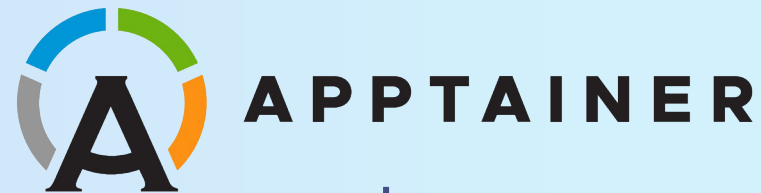
Storage & computing cost decrease

Docker VS Apptainer/Singularity

- When interacting with files on the host machine, Docker will try to work with maximum privilege whereas apptainer will always run at the user's privilege level.
- Apptainer natively supports GPU computation



Micro-services / Personal use



High Performance Computing

Apptainer definition

```
1 Bootstrap: docker
2 From: debian:stable-slim
3 Stage: devel
```

← A base image

A container's definition is the **recipe** to build a **container's image**

A list of Linux instructions to create the environment →

```
15 %post
16 # Libraries
17 # LAMMPS utils
18 apt-get update -y
19 apt-get -y install software-properties-common gpg-agent --no-install-recommends
20 apt-get install -y \
21     apt-utils \
22     build-essential
23 # add-apt-repository -y ppa:openkim/latest
24 apt-get update -y
25 apt-get upgrade -y
26 # apt-get install -y --no-install-recommends openmpi-bin python3 liblapack3 python3-venv libkim-api-dev openkim-models libpython3.6 \
27 #     hdf5-tools ffmpeg less libc6 libevent-2.1-7 libevent-pthreads-2.1-7 libexpat1 libfftw3-double3 libgcc-s1 libgomp1 libhwloc15 \
28 #     libjpeg-dev libltdl7 libopenmpi3 libpng16-16 libpython3.8 libstdc++6 libudev1 libvoro++1 libzstd1 zlib1g libreadline8 \
29 #     mpi-default-bin python3-dev python3-pip python3-pkg-resources python3-setuptools rsync ssh vim-nox valgrind gdb zstd \
30 #     libkim-api-dev openkim-models libopenmpi-dev gfortran && rm -rf /var/lib/apt/lists/*
31 apt-get install -y --no-install-recommends openmpi-bin python3 liblapack3 python3-venv libpython3.6 \
32     hdf5-tools ffmpeg less libc6 libevent-2.1-7 libevent-pthreads-2.1-7 libexpat1 libfftw3-double3 libgcc-s1 libgomp1 libhwloc15 \
33     libjpeg-dev libltdl7 libopenmpi3 libpng16-16 libpython3.8 libstdc++6 libudev1 libvoro++1 libzstd1 zlib1g libreadline8 \
34     mpi-default-bin python3-dev python3-pip python3-pkg-resources python3-setuptools rsync ssh vim-nox valgrind gdb zstd \
35     libopenmpi-dev gfortran
36
37 # Downloading and extracting sources
38 apt-get install -y wget
39 cd /opt
40 # MLIP + compilation
41 wget https://gitlab.com/ashapeev/mlip-2/-/archive/master/mlip-2-master.tar.gz
42 tar -xzf mlip-2-master.tar.gz
43 rm mlip-2-master.tar.gz
44 cd mlip-2-master
45 ./configure
46 make -j4 libinterface
```

Apptainer definition

More environment setup
to run the executable
Documentation on how to
use the container

```
%environment
export OMPI_MCA_plm_rsh_agent=
export LAMMPS_POTENTIALS=/usr/share/lammps/potentials

%test
lmp_mpi -h

%runscript
/usr/local/bin/lmp_mpi $*

%help
This container embedds LAMMPS (2 August 2023 stable version, update 2) with OpenMPI support and
MLIP support (https://gitlab.com/ashapeev/interface-lammps-mlip-2).
For more information about this image, please run "apptainer inspect <this-image>"

If you use "apptainer run <this-image>", the main executable "lmp_mpi" will be called.
For more information about it, please use "apptainer run <this-image> -h"
or "apptainer exec <this-image> lmp_mpi -h".

If you want to specify MPI number of processes, please use
"apptainer exec <this-image> mpirun -np <N> lmp_mpi ...".

You can also enter an interactive shell within the container with "apptainer shell <this-image>".

Finally, default interatomic potentials provided with this version of LAMMPS are available within
the container with the correct environment variables set to allow LAMMPS to locate them at runtime
(only used in the case where LAMMPS cannot locate the potential file in the location you specified)
```

Git to manage code files

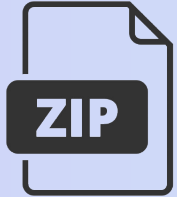
Main advantages of git

- Share codes easily
- Save storage by only saving modifications from the original file
- Allow you to go back to a specific version very easily

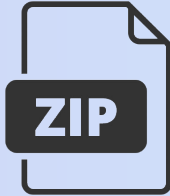
Git is a **free and open source** distributed **version control system** designed to handle everything from small to very large projects with speed and efficiency.

From git official website : git-scm.com

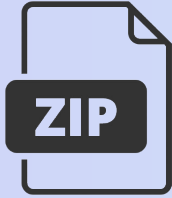
project-v1 : 18MB



project-v2 : 17MB



project-v3 : 18MB



project : 19MB



- v1 : 18MB
- v2 : -1MB
- v3 : +2MB

Why using git ?

- Used nearly everywhere (big & small companies, academia)
- Allow easier and faster collaboration
- Most niche open source software are only available through git
- Good free and easy way to distribute your scientific code
- Can also be used for all kinds of “evolving” files (personal notes, experiment data, thesis report, ...)

Working on software is like doing research, much of the work involves studying existing documentation.

If you run into trouble, your first step should be to check the docs.

<https://apptainer.org/docs/user/main/>

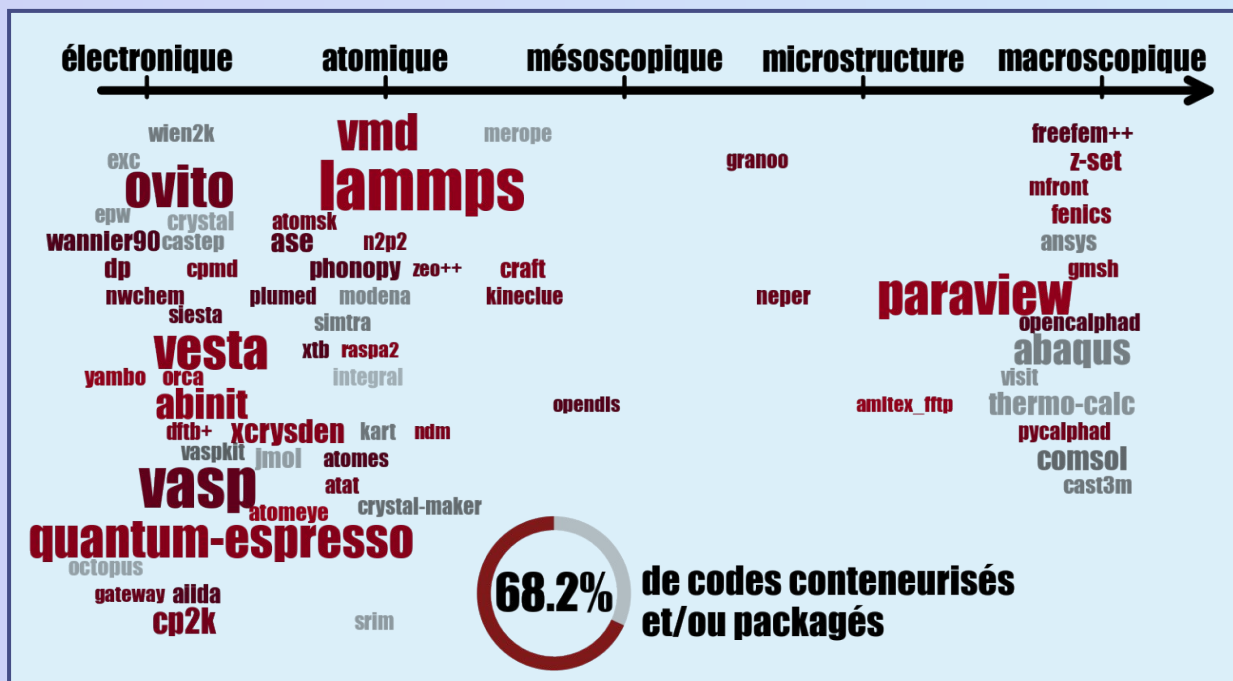
<https://git-scm.com/docs>

linux cheatsheet :

<https://linux-commands.labex.io>



<https://diamond-diadem.github.io/>



Welcome to the DIAMOND user documentation

❗ If you have any problems using the tools on the platform or if you have any queries, please get in touch with us via this [contact form](#).

The main purpose of this documentation is to help you get to grips with the various platform tools offered by **DIAMOND**.

Here are the links for quick access to the documentation section for the tools you are interested in:

- Installing Apptainer
- Installing Guix
- Using an Apptainer image
- Using a Guix package
- Frequently asked questions (FAQ)

Here are the links for quick access to image-specific tutorials:

Scientific computing	Visualisation
 AiiDA	 Ovito
 LAMMPS	 ParaView
 Quantum ESPRESSO	 VMD
 Abinit	
 FreeFEM	
 OpenCalphad	
 PLUMED	
 Gmsh	
 Zeo++	
 xTB	
 AMITEX_FFTF	
 NDM	
 GPUMD	

And more to be added!

And more to be added!



PROGRAMME
DE RECHERCHE
MATÉRIAUX
ÉMERGENTS

Thanks for your attention

pepr-diadem.fr



 **anr**®
agence nationale
de la recherche

cd [path] → move to a directory

ls [path] → list files in directory

touch [file] → create a new file

mkdir [directory] → create a directory

mv [files] [path] → move file to path

cp [files] [path] → copy files to path

cat [file] → display content of a file

rm [path] → delete files or directory

head/tail [file] → display start/end of a file

echo [string] → display line a text

clear → clear terminal output

grep [pattern] [file] → search for a specific pattern in file

-h --help

→ display information on the command

-r --recursive

→ use command on multiple files

--version

→ display current version of software



Options **change for every piece of software**, but some are commonly used.

The help argument will mainly work for every command. **man** command can be used if help does not provide help

Appendix – Linux shell operators

[content] > [file] → write content to file (use >> to append)

[command1] | [command2] → send output of command1 to command2

[command] & → run command without blocking the shell



Operators can be chained together as much as needed, allowing to create complex query with simple commands.

Appendix - Apptainer

apptainer **run** [image] → run user-defined command

apptainer **exec** [image] [command] → run command inside container

apptainer **build** [image] [def] → build container image from definition

apptainer **pull** [image] [url] → retrieve image from an url

apptainer **inspect** [image] → get metadata of an image

apptainer **run-help** [image] → display user-defined help of image

`git clone [url]` → clone a remote repository

`git add [file]` → add modified file to next commit

`git commit -m [message]` → stage previously add changes

`git push` → push local changes to remote repository

`git status` → informations on current git project

`git checkout [commit]` → change working branch to given commit